

PROGRAMACIÓN APLICADA 2026-2

CLASE 04

DER lógico v0.1, diccionario de datos, nulabilidad y restricciones

GUÍA DEL ESTUDIANTE

Observa ParkFlow 360, comprende el patrón y aplícalo a TU proyecto asignado

Duración	90 minutos
Proyecto espejo	ParkFlow 360
Frontera técnica	Diseño lógico; todavía sin CREATE TABLE
Evidencia de cierre	DER lógico + diccionario + decisiones + commit

1. Cómo debes usar esta guía

Regla principal

ParkFlow 360 NO es tu proyecto. El profesor lo utiliza como ejemplo. Tu obligación es identificar el método que está aplicando y producir el equivalente en el proyecto asignado a tu grupo. No copies nombres de entidades, columnas ni relaciones de ParkFlow si no existen realmente en tu dominio.

En esta clase trabajarás con el resultado de tu Clase 03: tu modelo relacional inicial. Ahora lo convertirás en un DER lógico v0.1 y en un diccionario de datos defendible.

2. Lo que debes lograr hoy

- Tener un DER lógico v0.1 de tu propio proyecto.
- Definir PK y FK del núcleo del dominio.
- Decidir qué atributos son obligatorios y cuáles pueden ser nulos.
- Definir unicidades de negocio y dominios de valores preliminares.
- Crear un diccionario de datos v0.1.
- Relacionar decisiones de integridad con las RN/RF de tu ficha oficial.
- Dejar evidencia en Git del trabajo del equipo.

Todavía NO corresponde

No escribas CREATE TABLE. No crees entidades JPA. No uses Hibernate. No intentes “probar” el modelo creando tablas manualmente. Primero debes ser capaz de justificar el diseño.

3. La dinámica: Profesor → Tu equipo

Cuando el profesor haga esto en ParkFlow...	Tu equipo debe hacer esto en su proyecto...
Define PK/FK de Zona y Espacio	Identificar PK/FK de las entidades nucleares de su propio dominio.
Decide si observación puede ser NULL	Justificar nulabilidad atributo por atributo en sus entidades.
Usa UNIQUE (zona_id, codigo)	Determinar si sus unicidades son globales o contextuales.
Relaciona RN con constraints	Tomar RN de su ficha y decidir cómo protegerlas.
Construye diccionario de Espacio	Construir diccionario de datos de sus tablas lógicas.
Hace commit del DER	Versionar los tres documentos de su propio proyecto.

4. Paso 1 — Recupera tus fuentes oficiales

Abre la ficha oficial del proyecto asignado y ten a mano, como mínimo: situación del cliente, actores, alcance funcional, reglas de negocio, modelo mínimo esperado, requisitos funcionales y flujo crítico. También abre tu model-relational-v0.1.md de la Clase 03.

5. Paso 2 — Convierte tu modelo relacional en DER lógico

Para cada tabla candidata, registra: nombre, PK, FK, atributos importantes, cardinalidades y restricciones candidatas. Tu DER lógico debe permitir explicar cómo una fila se relaciona con otra.

Ejemplo de patrón — NO copiar literalmente:

```
TABLA_A
PK tabla_a_id
NN nombre

TABLA_B
PK tabla_b_id
FK tabla_a_id -> TABLA_A.tabla_a_id
NN codigo
UQ (tabla_a_id, codigo)
```

6. Paso 3 — Decide la PK de cada tabla

Pregúntate: ¿qué identifica una fila de manera estable? En esta materia normalmente utilizaremos una PK técnica y, cuando exista, conservaremos la clave natural como UNIQUE.

Ejemplos de transferencia

En BiblioSmart, un código de ejemplar puede ser clave natural única. En CargoTrack, el tracking puede ser único. En AutoCare, la placa/VIN puede requerir unicidad. En tu proyecto, identifica la clave natural real; no inventes una sólo porque ParkFlow tiene placa.

7. Paso 4 — Define las FK y la optionalidad de cada relación

Para cada relación 1:N de la Clase 03, decide dónde debe vivir la FK. Luego responde si la FK puede ser NULL.

1. ¿Puede existir la entidad hija sin la entidad padre?
2. ¿La ficha del proyecto permite ese estado?
3. ¿La relación es obligatoria desde el momento de creación o puede completarse después?
4. ¿Qué dato quedaría inconsistente si la relación faltara?

8. Paso 5 — Decide la nulabilidad atributo por atributo

No uses “NOT NULL para todo”. Un atributo es obligatorio si el negocio no permite una instancia válida sin ese dato. Si puede faltar legítimamente, debe documentarse como opcional.

Atributo propio	¿Obligatorio?	¿Puede ser NULL?	Justificación de negocio
_____	Sí / No	Sí / No	_____
_____	Sí / No	Sí / No	_____
_____	Sí / No	Sí / No	_____
_____	Sí / No	Sí / No	_____

9. Paso 6 — Define unicidades correctamente

Una restricción UNIQUE responde: “¿qué combinación de datos no puede repetirse?”. Distingue entre unicidad global y contextual.

- Global: un código que no debe repetirse en todo el sistema.
- Contextual: un código que sólo debe ser único dentro de un padre, sucursal, ruta, catálogo u otro contexto.
- Compuesta: la unicidad depende de dos o más atributos.

10. Paso 7 — Identifica dominios y restricciones simples

Revisa estados, cantidades, fechas, rangos y valores permitidos. Marca restricciones conceptuales como: estado pertenece a un conjunto permitido; cantidad > 0; fecha_fin > fecha_inicio; porcentaje entre 0 y 100; código no vacío.

11. Paso 8 — Separa constraints estructurales de reglas transaccionales

No todas las RN se resuelven con PK/FK/UNIQUE/NOT NULL. Algunas dependen de varias filas, estados previos, rangos de fecha o permisos. Esas reglas deberán implementarse posteriormente en backend y, cuando sea posible, reforzarse en base de datos.

Ejemplos de tu banco de proyectos: impedir doble reserva, controlar transición de estados, validar stock suficiente, impedir devolución superior a lo vendido, cerrar una ruta sólo cuando corresponda. Debes clasificar las reglas de TU ficha.

12. Paso 9 — Crea el diccionario de datos v0.1

Para cada tabla lógica prioritaria, documenta por lo menos:

Campo	Qué debes escribir
Nombre del atributo	Nombre consistente y entendible.
Significado	Qué representa en el dominio.
Obligatorio	Sí/No y por qué.
PK/FK/UQ	Rol estructural del atributo.
Dominio/regla	Valores permitidos o condición relevante.
Origen	RN/RF/caso de uso que justifica su existencia.

13. Paso 10 — Archivos obligatorios de la clase

docs/04-model/der-logico-v0.1.md

docs/04-model/diccionario-datos-v0.1.md

docs/04-model/decisiones-integridad-v0.1.md

Plantilla de der-logico-v0.1.md:

```
# DER lógico v0.1 — [NOMBRE DE TU PROYECTO]
```

```
## Convenciones
```

- PK = clave primaria
- FK = clave foránea
- UQ = unicidad
- NN = obligatorio

```
## [tabla_1]
PK ...
NN ...
UQ ...
```

```
## [tabla_2]
PK ...
FK ... -> ...
NN ...
```

```
## Relaciones
1. [A] 1 ---- N [B]
2. ...
```

```
## Reglas que afectan el modelo
- RN-__: ... -> decisión: ...
- RN-__: ... -> decisión: ...
```

Plantilla de decisiones-integridad-v0.1.md:

```
# Decisiones de integridad v0.1

| RN/RF | Regla | Protección prevista | Justificación |
|---|---|---|---|
| RN-__ | ... | PK/FK/UQ/NN/CHECK/backend | ... |
| RN-__ | ... | ... | ... |
```

14. Ejercicio obligatorio: prueba de contradicción

Elige tres reglas de negocio de tu ficha. Para cada una, inventa un dato inválido que podría entrar si tu diseño no tuviera protección. Luego indica qué mecanismo lo evitará.

Regla de tu proyecto	Estado inválido posible	Protección prevista
RN-__	_____	_____
RN-__	_____	_____
RN-__	_____	_____

15. Revisión cruzada del equipo

Un integrante que NO haya redactado la tabla debe revisarla y contestar:

- ¿Entiendo qué representa cada atributo?
- ¿Puedo justificar por qué la FK está en ese lado?
- ¿Hay una columna nullable que debería ser obligatoria?

- ¿Hay una columna obligatoria que legítimamente puede faltar?
- ¿La unicidad tiene el ámbito correcto?
- ¿El modelo contradice alguna RN o flujo crítico?

16. Git — evidencia obligatoria

```
git status
git add docs/04-model
git commit -m "docs: consolidar DER logico y diccionario de datos v0.1"
git log --oneline -5
```

17. Qué debe poder defender cualquier integrante

5. ¿Cuál es la PK de una tabla importante y por qué?
6. ¿Qué clave natural existe y cómo se protege?
7. ¿Dónde está una FK importante y por qué está en ese lado?
8. ¿Qué atributo puede ser NULL y cuál no? Justifica ambos.
9. Da un ejemplo de UNIQUE global y/o compuesto de tu proyecto.
10. ¿Qué RN no puede resolverse solamente con constraints simples?
11. ¿Cómo se conecta tu DER con el flujo crítico del proyecto?

18. Errores que harán que el trabajo sea devuelto

- Copiar el DER de ParkFlow cambiando nombres.
- Usar tablas que no aparecen ni se justifican por el dominio de tu ficha.
- No indicar PK o FK.
- Poner todos los atributos como obligatorios sin análisis.
- No distinguir unicidad global de contextual.
- Tener un diccionario que sólo repite el nombre de la columna.
- No relacionar decisiones con RN/RF.
- Hacer cambios sin commit o sin que otro integrante pueda explicarlos.

19. Checklist de cierre de tu grupo

- ☐ DER lógico v0.1 creado para NUESTRO proyecto.
- ☐ PK/FK del núcleo definidas.
- ☐ Nulabilidad principal justificada.
- ☐ Unicidades identificadas.
- ☐ Dominios/restricciones preliminares listados.
- ☐ Diccionario de datos v0.1 creado.
- ☐ Al menos 3 RN/RF trazadas a decisiones de integridad.
- ☐ Revisión cruzada realizada.
- ☐ Commit hecho y verificable.

20. Autoevaluación individual

Marca mentalmente VERDE si puedes explicarlo sin leer, AMARILLO si necesitas apoyo y ROJO si todavía no sabes hacerlo: PK, FK, UNIQUE, NULL, relación 1:N, unicidad compuesta, dominio controlado, regla transaccional y diccionario de datos. Todo ROJO debe convertirse en una duda concreta antes de la próxima clase.

21. Definition of Done — Clase 04

Tu equipo termina la clase cuando...

Existe un DER lógico v0.1 coherente con su proyecto asignado, existe un diccionario de datos que explica el significado y las restricciones principales, las reglas relevantes están trazadas a mecanismos de integridad y cualquier integrante puede defender las decisiones principales.

22. Preparación para la Clase 05

En la próxima clase convertirás este diseño lógico en una especificación preparada para PostgreSQL: nombres físicos, tipos de datos preliminares, estrategia de identificadores, timestamps, precisión numérica, constraints y estructura de la primera migración. Si tu DER de hoy está mal, el SQL de mañana heredaré ese error.